Exception and Exception Handler

Exception

- An exception is a unusual, often unpredictable event, detectable by software or hardware, that requires special processing occurring at runtime
- In C++, a variable or class object that represents an exceptional event.

Handling Exception

- If without handling,
  - Program crashes
  - Falls into unknown state
- An exception handler is a section of program code that is designed to execute when a particular exception occurs
  - Resolve the exception
  - Lead to known state, such as exiting the program

Standard Exceptions

- Exceptions Thrown by the Language
- Exceptions Thrown by Standard Library Routines
- Exceptions Thrown by user code, using *throw* statement

The *throw* Statement

Throw: to signal the fact that an exception has occurred; also called *raise*

ThrowStatement

```
throw  Expression
```

The try-catch Statement

How one part of the program catches and processes the exception that another part of the program throws.

TryCatchStatement

```
try
   Block
catch (FormalParameter)
   Block
catch (FormalParameter)
```

FormalParameter

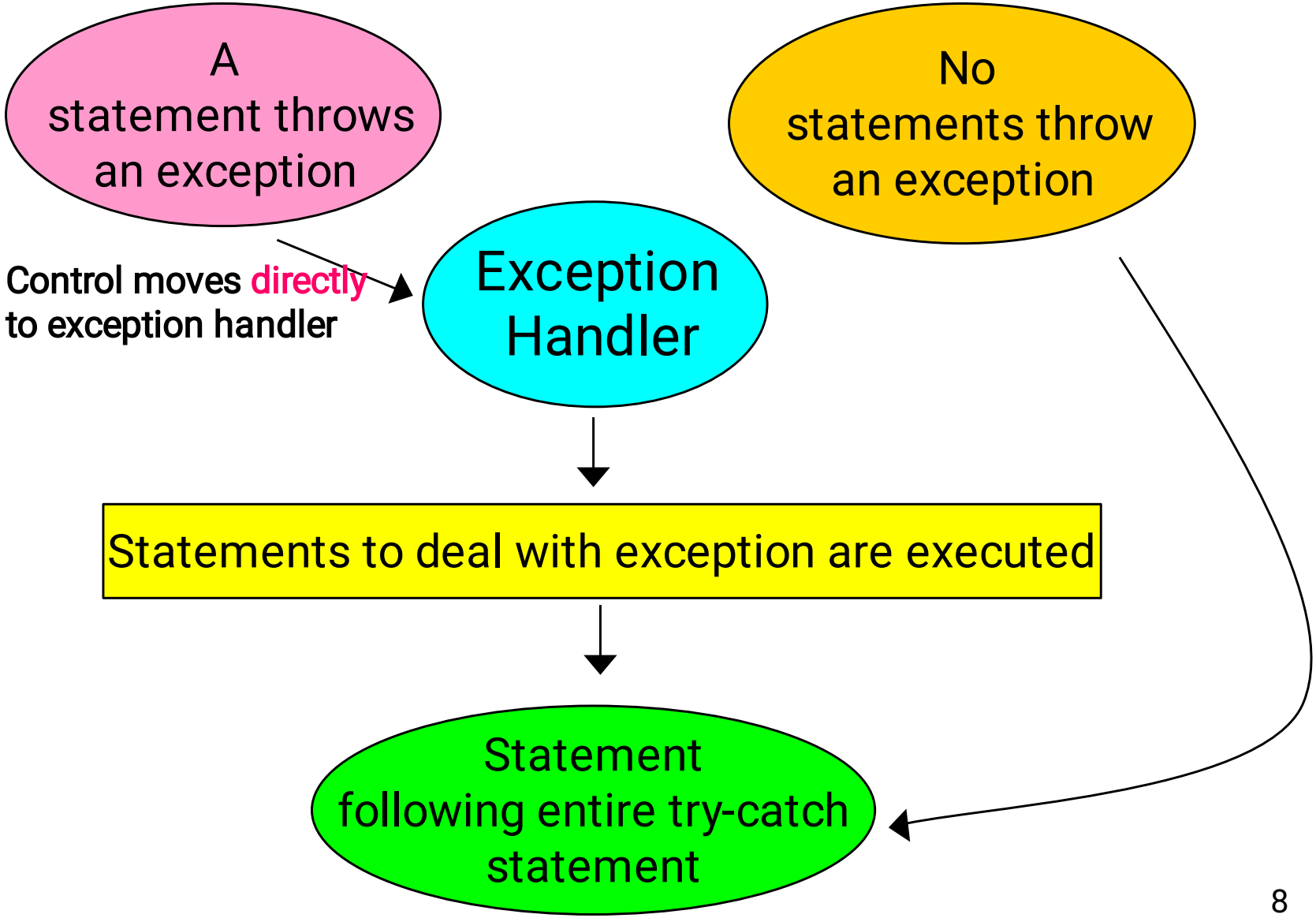
```
DataType VariableName

...
```

Example of a try-catch Statement

```
try
{
        // Statements that process personnel data and may throw
        // exceptions of type int, string, and SalaryError
}
catch ( int )
{
        // Statements to handle an int exception
}
catch ( string s )
{
       cout << s << endl;  // Prints "Invalid customer age"
        // More statements to handle an age error
}
catch ( SalaryError )
{
       // Statements to handle a salary error
}
```

Execution of try-catch



Throwing an Exception to be Caught by the Calling Code

```
void Func3()
{

    try
    {

        Func4();

    }
    catch ( ErrType )
    {

    }

}
```

Function call

Normal return

```
void Func4()
{

  if ( error )
     throw ErrType();

}
```

Return from thrown exception

Practice: Dividing by ZERO

Apply what you know:

```
int Quotient(int numer,    // The numerator
        int denom )   // The denominator
{
   if (denom != 0)
      return numer / denom;
   else
      //What to do?? do sth. to avoid program    //crash
}
```

Example: Handling an Attempt to Divide by Zero

```
// Fig. 27.1: DivideByZeroException.h
// Class DivideByZeroException definition.
#include <stdexcept> // stdexcept header file contains runtime_error
using std::runtime_error; // standard C++ library class runtime_error

// DivideByZeroException objects should be thrown by functions
// upon detecting division-by-zero exceptions
class DivideByZeroException : public runtime_error
{
public:
   // constructor specifies default error message
   DivideByZeroException::DivideByZeroException()
      : runtime_error( "attempted to divide by zero" ) {}
}; // end class DivideByZeroException
```

Zero Divide Example

- Fig27-2
  - (1 of 2)

```cpp
// Fig. 27.2: Fig27_02.cpp
// A simple exception-handling example that checks for
// divide-by-zero exceptions.
#include <iostream>
using std::cin;
using std::cout;
using std::endl;

#include "DivideByZeroException.h" // DivideByZeroException class

// perform division and throw DivideByZeroException object if
// divide-by-zero exception occurs
double quotient( int numerator, int denominator )
{
   // throw DivideByZeroException if trying to divide by zero
   if ( denominator == 0 )
      throw DivideByZeroException(); // terminate function

   // return division result
   return static_cast< double >( numerator ) / denominator;
} // end function quotient

int main()
{
   int number1; // user-specified numerator
   int number2; // user-specified denominator
   double result; // result of division

   cout << "Enter two integers (end-of-file to end): ";
```

Zero Divide Example

- Fig27-2
  - (2 of 2)

```
   // enable user to enter two integers to divide
   while ( cin >> number1 >> number2 )
   {
      // try block contains code that might throw exception
      // and code that should not execute if an exception occurs
      try
      {
         result = quotient( number1, number2 );
         cout << "The quotient is: " << result << endl;
      } // end try

      // exception handler handles a divide-by-zero exception
      catch ( DivideByZeroException &divideByZeroException )
      {
         cout << "Exception occurred: "
            << divideByZeroException.what() << endl;
      } // end catch

      cout << "\nEnter two integers (end-of-file to end): ";
   } // end while

   cout << endl;
   return 0; // terminate normally
} // end main
```